Assignment-19.1

HT No:2303A51585

Batch No:28

Course:AI Assisted Coding

Task 1 – Python to Java Conversion

PROMPT:

Create a simple Python class with attributes and methods, then convert it into an equivalent Java class. Include constructor, proper data types, and method signatures for each and compare the results of both versions

CODE:

lab_19-1.py > ...

```python
#Create a simple Python class with attributes and methods, then convert it into an equivalent Java class.
#Include constructor, proper data types, and method signatures for each and compare the results of both versions

Windsurf: Refactor | Explain
class Person:
    Windsurf: Refactor | Explain | Generate Docstring |
    def __init__(self, name, age):
        self.name = name
        self.age = age

    Windsurf: Refactor | Explain | Generate Docstring |
    def greet(self):
        print(f"Hello, my name is {self.name} and I am {self.age} years old.")

person = Person("John Doe", 30)
person.greet()
```

```
Windsurf: Refactor | Explain
class Person {
    public String name;
    public int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    Windsurf: Refactor | Explain | Generate Javadoc | ×
    public void greet() {
        System.out.println("Hello, my name is " + name + " and I am " + age + " years old.");
    }
}

Windsurf: Refactor | Explain
public class Main {
    Run | Debug | Windsurf: Refactor | Explain | Generate Javadoc | ×
    public static void main(String[] args) {
        Person person = new Person(name: "John Doe", age: 30);
        person.greet();
    }
}
```

OUTPUT:

```
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding> ^C
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding> & C:/Users/Pavani/AppData/Local/Micros
ani/OneDrive/Documents/AI Assisted coding/lab_19-1.py"
Hello, my name is John Doe and I am 30 years old.
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding>
```

```
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding> javac Main.java
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding> java Main
Hello, my name is John Doe and I am 30 years old.
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding>
```

EXPLANATION:

- Python is dynamically typed (no need to declare types), while Java is statically typed (String, int, etc. required).
- Python uses __init__, whereas Java uses a constructor with the class name.
- Python is indentation-based and simpler; Java uses {} and ; with stricter structure.
- Python runs directly, while Java needs compilation (javac) before running (java).

Task 2 – C++ to Python Function Conversion

PROMPT:

Convert a recursive C++ function for calculating factorial into Python. Make sure the Python code is correct, follows Python conventions, and include a few test cases to verify the output

CODE:

lab_19-1.py > ...

```python
# Convert a recursive C++ function for calculating factorial into Python.
# Make sure the Python code is correct, follows Python conventions, and include a few test cases to verify the output
Windsurf: Refactor | Explain | Generate Docstring | X
def factorial(n):
    if n < 0:
        raise ValueError("Factorial is not defined for negative numbers")
    elif n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)
# Test cases
print(factorial(0))  # Output: 1
print(factorial(1))  # Output: 1
print(factorial(5))  # Output: 120
try:
    print(factorial(-3))  # This should raise an error
except ValueError as e:
    print(e)  # Output: Factorial is not defined for negative numbers
```

OUTPUT:

```
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding> & C:/Users/Pavani/AppData/Loc
ani/OneDrive/Documents/AI Assisted coding/lab_19-1.py"
1
1
120
Factorial is not defined for negative numbers
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding>
```

EXPLANATION:

- Convert the recursive C++ factorial function into Python while preserving the same logic and recursion flow
- Replace C++-specific elements (data types, braces, semicolons) with Python's simpler, indentation-based syntax.
- Test the Python function with sample inputs to ensure correctness and consistency with expected outputs.

Task 3 – SQL to Pandas Query

PROMPT:

SELECT name, salary FROM employees WHERE salary > 50000 convert the above sql query into an equivalent pandas code and test the code on sample dataFrame

CODE:

```
#SELECT name, salary FROM employees WHERE salary > 50000
#convert the above sql query into an equivalent pandas code and test the code on sample dataFrame
import pandas as pd
# Sample DataFrame
data = {
    'name': ['Alice', 'Bob', 'Charlie', 'David'],
    'salary': [60000, 45000, 70000, 30000]
}
df = pd.DataFrame(data)
# Equivalent pandas code to filter employees with salary greater than 50000
filtered_df = df[df['salary'] > 50000][['name', 'salary']]
print(filtered_df)
```

OUTPUT:

```
2  Charlie   70000
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding> & C:/Users/Pavani/AppData/Local/Microsoft/WindowsApp
lab_19-1.py"
      name  salary
0    Alice   60000
2  Charlie   70000
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding>
```

EXPLANATION:

- The SQL query is translated into Pandas by using boolean indexing to filter rows and selecting specific columns.
- A sample dataset is created using a dictionary and converted into a DataFrame for testing.
- The condition df['salary'] > 50000 filters employees with salaries greater than 50,000.
- The column selection ensures only name and salary are displayed, similar to the SQL query.

Task 4 – Real-Time Application: Multi-Language Snippet Converter

PROMPT:

Choose a real-world use case and write code in one programming language. Then convert the same code into another programming language. Ensure both versions have the same functionality and output.Also include sample inputs, outputs

CODE:

```
# Choose a real-world use case and write code in one programming language.
# Then convert the same code into another programming language.
# Ensure both versions have the same functionality and output.
# Also include sample inputs, outputs
tasks = []

Windsurf: Refactor | Explain | Generate Docstring | ×
def add_task(task):
    tasks.append(task)

Windsurf: Refactor | Explain | Generate Docstring | ×
def show_tasks():
    for i, task in enumerate(tasks, 1):
        print(f"{i}. {task}")

# Sample usage
add_task("Study AI")
add_task("Complete assignment")
add_task("Go for walk")

show_tasks()

#javascript code
# let tasks = [];

# function addTask(task) {
#     tasks.push(task);
# }

# function showTasks() {
#     tasks.forEach((task, index) => {
```

```javascript
// javascript code
let tasks = [];

Windsurf: Refactor | Explain | Generate JSDoc | ×
function addTask(task) {
    tasks.push(task);
}

Windsurf: Refactor | Explain | Generate JSDoc | ×
function showTasks() {
    tasks.forEach((task, index) => {
        console.log(`${index + 1}. ${task}`);
    });
}

// Sample usage
addTask("Study AI");
addTask("Complete assignment");
addTask("Go for walk");

showTasks();
```

OUTPUT:

```
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding> & C:/Users/Pavani/AppData/Local/Microsoft
ani/OneDrive/Documents/AI Assisted coding/lab_19-1.py"
1. Study AI
2. Complete assignment
3. Go for walk
2. Complete assignment
3. Go for walk
```

```
TERMINAL
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding> node lab19.js
1. Study AI
2. Complete assignment
3. Go for walk
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding>
```

EXPLANATION:

- A real-world use case was selected where the same functionality was implemented in two different programming languages.
- AI was used to translate the code from one language to another while maintaining the same logic and output.
- Both implementations were executed and tested to ensure they produced identical results.
- During translation, challenges such as differences in syntax, libraries, and execution environments were identified and handled.